Accident Detection and GPS Alert System

Smart Emergency Response Using ESP32, MPU-9250, and Telegram Bot

Sagar Kumar (225CS1026)
Akshay Singh (225CS2001)
Project done under the supervision of
Dr. Dev Narayan Yadav

Department of Computer Science And Engineering

National Institute of Technology Rourkela

Outline

Introduction

Overview

Road accidents are a leading cause of deaths worldwide. Emergency response is often delayed because no one reports the incident in time.

- Automated accident detection using IoT sensors
- Real-time GPS location tracking
- Instant Telegram alert to emergency contacts
- Low-cost, embedded system solution

Core Idea

If a victim is unconscious, the system automatically detects the crash and sends the exact GPS location to rescue contacts via Telegram Bot.

Problem Statement

- Delayed medical assistance after road accidents
- Victims may be unconscious and unable to call for help
- No real-time automatic accident notification system
- Difficulty in identifying the exact accident location

Impact

- Loss of life due to delayed emergency response
- Confusion during accident reporting
- Inaccessible or unknown accident sites

Objectives

Primary Objectives

- Detect vehicle accidents automatically using sensors
- Obtain real-time GPS coordinates of the accident site
- Send emergency alerts instantly via Telegram Bot
- Reduce response time during critical situations

Secondary Objectives

- Build a low-cost embedded safety system
- Use IoT for real-time monitoring and alerting
- Improve system reliability and sensor accuracy

System Overview

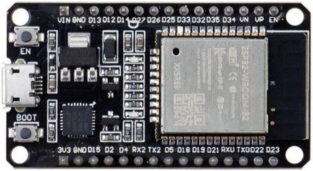
Working Concept

The ESP32 reads MPU-9250 data continuously. On detecting abnormal acceleration beyond a threshold, it fetches GPS coordinates and sends a Telegram alert.

Step-by-Step Flow

1. MPU-9250 detects sudden impact / abnormal motion
2. ESP32 checks if G-force exceeds accident threshold
3. GPS module fetches latitude and longitude
4. ESP32 connects to Wi-Fi and calls Telegram Bot API
5. Emergency contact receives alert with Google Maps link

Hardware: ESP32 NodeMCU



ESP32 NodeMCU – Main Controller

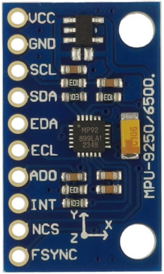
The **brain** of the system. Dual-core 32-bit MCU with built-in Wi-Fi and Bluetooth.

- Clock: **240 MHz** dual-core processor
- Built-in **Wi-Fi 802.11 b/g/n**
- 30 GPIO pins (I^2C, UART, SPI)
- Micro-USB for programming
- 3.3V logic, runs Arduino C++ firmware

Role in Project

Reads sensor data, runs detection logic, connects to Wi-Fi, and sends Telegram alert via HTTP GET request.

Hardware: MPU-9250 Sensor



MPU-9250 – 9-Axis IMU

Combines accelerometer, gyroscope, and magnetometer on a single chip.

- **3-axis Accelerometer** – G-force ($\pm$16g range)
- **3-axis Gyroscope** – angular velocity
- **3-axis Magnetometer** – orientation
- Interface: **I^{2}C** at address 0x68
- Pins: SDA$\rightarrow$GPIO21, SCL$\rightarrow$GPIO22, VCC$\rightarrow$3.3V

Role in Project

Detects sudden impact when G-force $> 2.5g$, triggering the accident alert sequence.

Hardware: GPS Module (Ublox 7M)

GPS Module – Ublox 7M with Compass

Compact GPS receiver with built-in ceramic antenna and compass.

- Chipset: **Ublox 7M** with integrated compass
- Position accuracy: $< 2.5\,\text{m}$ CEP
- Interface: **UART** at 9600 baud
- Pins: TX$\rightarrow$GPIO16, RX$\rightarrow$GPIO17
- Supply: 3.3V–5V
- Parsed with **TinyGPS++ library**

Role in Project

Provides latitude & longitude sent as a clickable Google Maps link in the Telegram alert.

Software Requirements

Software Tools

- Arduino IDE
- Telegram Bot API
- TinyGPS++ Library
- MPU9250 Library
- WiFi / HTTPClient Library

Programming

- Embedded C / Arduino C++
- REST API (HTTP GET/POST)
- JSON parsing for Telegram

Telegram Bot Setup

A bot is created using **BotFather** on Telegram. The ESP32 sends HTTP GET requests to the Telegram API with the emergency message and GPS coordinates.

Telegram Bot Setup

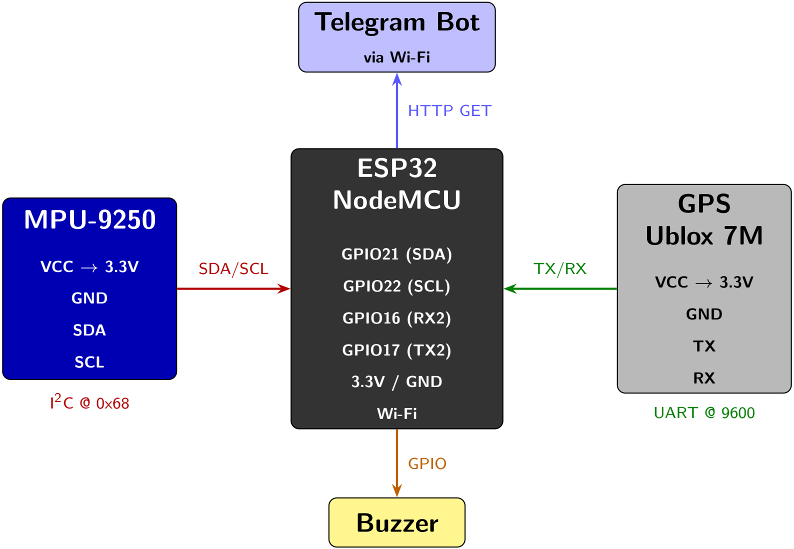
Steps to Create the Bot

1. Open Telegram, search **BotFather**
2. Send `/newbot` command
3. Enter bot name: `NITR CSE BOT`
4. Enter username: `NITRCSBot`
5. BotFather returns a unique **Bot Token**
6. Get **Chat ID** using IDBot or getUpdates API
7. Paste Token & Chat ID into ESP32 firmware

Our Bot

- Name: **NITR CSE**
- Username: `@NITRCSBot`
- API: Telegram HTTP Bot API
- Library: UniversalTelegramBot

Circuit Diagram



Working Principle

Accident Detection Logic

The MPU-9250 measures acceleration on all three axes. The resultant magnitude is computed and compared to a threshold:

$$G_{mag} = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

If $G_{mag} > G_{threshold}$, an accident is detected.

Threshold Tuning

- Normal driving: $\approx 1g$ (gravity only)
- Speed bumps / potholes: $\approx$ 2–3g
- Accident / impact: $> 2.5g$ (set in firmware)

Telegram Alert Message

Message Format Sent on Detection

```
[ALERT] Accident Detected!
Vehicle may be involved in an accident.
[LOC] Latitude:  XX.XXXXXX   Longitude:  YY.YYYYYY
[MAP] https://maps.google.com/?q=LAT,LONG
```

Features

- Instant Telegram notification
- Clickable Google Maps link
- 6-decimal GPS precision

Alert Trigger

Alert is sent **immediately** when G-force exceeds 2.5g and GPS fix is valid. No delay or manual intervention required.

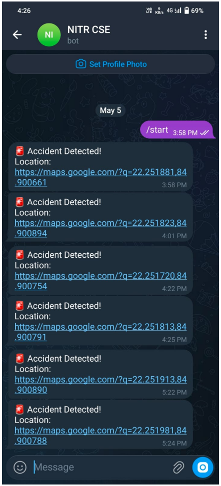
Code: Accident Detection & Alert

```
// Read MPU-9250 accelerometer via I2C
Wire.beginTransmission(0x68);
Wire.write(0x3B);
Wire.endTransmission(false);
Wire.requestFrom(0x68, 6, true);
ax = Wire.read()<<8 | Wire.read();
ay = Wire.read()<<8 | Wire.read();
az = Wire.read()<<8 | Wire.read();

// Calculate total acceleration magnitude
float accX = ax / 16384.0;   // Convert raw to g
float accY = ay / 16384.0;
float accZ = az / 16384.0;
totalAcc = sqrt(accX*accX + accY*accY + accZ*accZ);

// Check accident threshold (2.5g)
if (totalAcc > 2.5) {
  if (gps.location.isValid()) {
    String msg = "Accident Detected!\nLocation:\n";
    msg += "https://maps.google.com/?q=";
    msg += String(gps.location.lat(), 6) + ",";
    msg += String(gps.location.lng(), 6);
    bot.sendMessage(CHAT_ID, msg, "");
    Serial.println("Alert Sent!");
  }
}
```

Key Points

Threshold = **2.5g** | Instant alert on detection | GPS validity checked before sending

Results and Observations



Live Test: NITRCSBot

- **6 alerts** delivered successfully
- GPS near NIT Rourkela campus
- Coords: `22.2518, 84.9006`
- Accuracy: <5 m variation
- Response time: **2–5 seconds**

Test Observations

- Google Maps link clickable in chat
- Triggered on manual shake test
- All alerts received with timestamp

Known Limitations

- Requires active Wi-Fi connection
- GPS needs open sky for first fix

Future Enhancements

- GSM module (SIM800L) support for areas without Wi-Fi
- Integration with cloud dashboard for live monitoring
- AI-based false positive filtering using ML models
- Mobile application for real-time tracking
- Camera module for accident video recording
- Automatic ambulance / hospital notification

Vision

A fully autonomous smart vehicle safety system that operates without human intervention and integrates directly with emergency services.

Conclusion

Summary

The Accident Detection and GPS Alert System combines embedded IoT hardware with real-time communication to improve emergency response during road accidents.

- ESP32 + MPU-9250 detect crash events reliably
- GPS module provides accurate location data
- Telegram Bot delivers instant alerts to contacts
- Low-cost and easily deployable in real vehicles

Key Takeaway

This project demonstrates how IoT and embedded systems can directly contribute to road safety and save lives.

References

1. ESP32 Technical Reference Manual – Espressif Systems
2. InvenSense MPU-9250 Product Specification, Rev 1.1
3. u-blox 7M GPS Module Datasheet – u-blox AG
4. Telegram Bot API Documentation – `core.telegram.org/bots/api`
5. TinyGPS++ Library – Mikal Hart, GitHub
6. Ngu et al., "IoT-based Accident Detection and Alert System", *IEEE Access*, 2021

Thank You!

Questions and Discussion

"Technology for Safer Roads and Faster Emergency Response"